

Тестовое задание: сервис сопоставления должностей с классификатором

Ориентировочное время: 3-4 часа. Мы не ждем идеального продукта. Нам интересен ваш подход, выбор инструментов и умение его обосновать.

Контекст

В строительную платформу поступают данные из внешних систем (например, из 1С). Наименования должностей в них написаны как угодно: с сокращениями, опечатками, лишними словами, синонимами. Чтобы данные можно было анализировать, каждое наименование нужно привести к системному классификатору. Сейчас это делается вручную. Ваша задача: автоматизировать сопоставление.

Данные (в папке data)

- classifier.csv: классификатор должностей, 56 записей (код; каноническое наименование).
- raw_positions.csv: 300 исходных наименований из внешней системы. Часть сопоставляется с классификатором, часть не имеет соответствия (например, офисные должности).
- labeled_sample.csv: 50 записей с правильными ответами. Используйте их для оценки качества вашего решения. Для записей без соответствия правильный ответ: НЕТ СООТВЕТСТВИЯ.

Кодировка файлов UTF-8, разделитель точка с запятой.

Что нужно сделать

1. Сервис или скрипт на Python, который для каждой записи из raw_positions.csv возвращает: код по классификатору (или НЕТ СООТВЕТСТВИЯ), наименование по классификатору и степень уверенности (от 0 до 1).
2. Записи с низкой уверенностью должны помечаться для ручной проверки. Порог выберите и обоснуйте сами.
3. Оцените качество своего решения на labeled_sample.csv и приведите метрику (например, точность).
4. Результат сопоставления сохраните в results.csv (id; исходное наименование; код; наименование по классификатору; уверенность; требует проверки да/нет).

Что прислать

- Код (репозиторий или архив) с инструкцией запуска (README).
- results.csv с результатами по всем 300 записям.
- Короткую записку (примерно 1 страница): какой подход выбрали и почему; какие подходы отвергли и почему; что бы изменили, если бы записей было 3 миллиона, а классификатор насчитывал 10 тысяч значений; как бы улучшали качество дальше.

Ограничения и подсказки

- стек свободный, но только open source. Обращения к внешним платным API (OpenAI и т.п.) использовать нельзя: в продакшене решение работает в закрытом контуре.
- Использовать LLM или эмбединги не обязательно. Выбирайте инструменты под задачу, а не наоборот. Простое и точное решение лучше сложного и модного.
- Разряды и категории (например, 5 разряда) не влияют на соответствие должности.
- Решение должно запускаться одной-двумя командами на обычной машине без GPU.

На что мы смотрим

- Качество сопоставления на скрытой разметке (у нас есть полные правильные ответы).
- Обоснованность выбора подхода и честная оценка его слабых мест.
- Читаемость кода и воспроизводимость запуска.
- Работу с уверенностью: понимание, когда решению нельзя доверять.

Вопросы по заданию приветствуются: умение задать правильный вопрос мы тоже ценим. Удачи!